Source Code

Ruby executes programs from plain old files. You can look at these files to examine the source code that makes up your program using one of a number of techniques.

The special variable __FILE__ contains the name of the current source file. This leads to a fairly short (if cheating) quine—a program that outputs its own source code:

```
print File.read(__FILE__)
```

produces:

```
print File.read(__FILE__)
```

As we saw in the previous section, the method caller returns the call stack as a list. Each entry in this list starts off with a filename, a colon, and a line number in that file. You can parse this information to display source. In the following example, we have a main program, main.rb, that calls a method in a separate file, sub.rb. That method in turns invokes a block, where we traverse the call stack and write out the source lines involved. Notice the use of a hash of file contents, indexed by the filename.

Here's some code that dumps out the call stack, including source information:

ospace/caller/stack_dumper.rb

```
def dump_call_stack
  file_contents = {}
  puts "File            Line  Source Line"
  puts "---------------+----+------------"
  caller.each do |position|
    match_data = position.match(/\A(.*?):(\d+)/)
    next if match_data.nil?
    file = match_data[1]
    line = Integer(match_data[2])
    file_contents[file] ||= File.readlines(file)
    printf("%-15s:%3d - %s",
      File.basename(file),
      line,
      file_contents[file][line - 1].lstrip)
  end
end
```

The file sub.rb contains a single method:

ospace/caller/sub.rb

```
def sub_method(v1, v2)
  main_method(v1 * 3, v2 * 6)
end
```

The following is the main program, which invokes the stack dumper after being called back by the submethod:

ospace/caller/main.rb

```
require_relative "sub"
require_relative "stack_dumper"

def main_method(arg1, arg2)
  dump_call_stack
end
```